Advance Java LAB

Rajasthan Technical University, Kota (Raj.)

Software Requirements Specification(SRS)

Project Name-
CampusCare

Submitted By:

1. Roshan Jat	24EARAD143
2. Sufiyan Khan	24EARAD156
3. Sufiyan Nagori	24EARAD157
4. Sumit Kumar	24EARAD158
5. Divyanshu Geel	25EARAD702

Under the supervision of

Er. Ram Babu Buri

Assistant Professor

Session: 2025-26

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

Document Control and Table of Contents

Version	Date	Author	Change
1.0	13 October 2026	Project team	First complete specification

Section	Heading	Core topics covered
1	Introduction	Purpose, scope, definitions and acronyms, references, overview
2	Overall Description	Product perspective, product functions, user classes, operating environment, constraints, user documentation, assumptions and dependencies
3	Specific Requirements	3.1 External interfaces; 3.2 Functional requirements for modules M1 to M7 and core; 3.3 Use cases; 3.4 Non-functional requirements; 3.5 Database requirements; 3.6 Business rules
4	Appendices	A Traceability matrix; B Status transition table; C Assumptions; D Seeded master data; E Suggested diagrams

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of CampusCare, the Smart Campus Issue Management and Resolution System. It describes what the system shall do, who uses it, the rules it enforces and the constraints under which it is built. The document is the agreed basis for design, implementation, testing and evaluation of the project, and follows the structure of IEEE Std 830-1998 and ISO/IEC/IEEE 29148.

Campus issues such as Wi-Fi failures, electrical faults, library, hostel, transport, security and furniture problems are currently reported by phone calls, paper applications or word of mouth. There is no tracking, no accountability, no history and no analytics, so complaints are lost or delayed and students never learn their status. CampusCare gives the campus one digital channel to report an issue, routes and assigns it automatically to the responsible department, lets staff resolve it with remarks, keeps a complete history, notifies users on every change, lets students rate the resolution or reopen the complaint, and gives the administrator analytics and reports.

Objectives

- one digital channel for all campus complaints;
- automatic routing and assignment of each complaint to the responsible department;
- real-time status tracking with a complete audit history;
- notifications on every status change;
- student feedback, department ratings and a reopen option;
- admin analytics and CSV reports to improve service quality.

1.2 Scope

CampusCare is a Windows desktop application. Each user runs a JavaFX client on their own computer, and all clients work on one shared cloud PostgreSQL database, so that students, staff and the administrator on different computers see the same data. The system will:

- register students, authenticate all users and open a dashboard that matches the role (Student, Staff, Admin);
- let students submit complaints with category, location, priority, description and one optional attachment;
- take the department from the category and automatically assign each complaint to the active staff member of that department with the lowest workload;
- let the administrator assign or re-assign complaints, and let the assigned staff member accept, start work and resolve them;
- record every status change in a complaint history and show it as a timeline;
- create in-app notifications for submission, assignment, every staff action, reopening and closure;
- let students give feedback, which closes the complaint, or reopen a resolved complaint;
- give the administrator analytics, feedback and department ratings, management screens for users, departments, categories and locations, and CSV export of complaints.

The following are not part of this version: web or mobile clients, e-mail or SMS notifications (notifications are shown inside the application only), integration with other college systems, and operation without an internet connection.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
BR / UC	Business Rule / Use Case
RBAC	Role-Based Access Control: what a user may do depends on the user's role
JavaFX / FXML	Java toolkit for desktop interfaces / XML files that describe JavaFX screens
JDBC	Java Database Connectivity, the Java API used to talk to the database
PostgreSQL / Neon	Relational database system / the cloud service that hosts the shared database
BCrypt	One-way, salted password hashing algorithm
DAO	Data Access Object: a class that holds the database access code for one entity
Maven / jpackage	Build tool / tool that creates the Windows EXE with a bundled Java runtime
Complaint	A campus issue recorded in the system
Category	Type of complaint; each category belongs to exactly one department
Department	Campus unit that resolves complaints of its categories (for example IT, Electrical)
Master data	Departments, categories and locations used to fill selection lists
History / audit trail	Chronological record of every status change of a complaint
Transaction	A group of database changes that are saved together or not at all
CSV	Comma-separated values file that can be opened in a spreadsheet

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148, Systems and software engineering: Requirements engineering (structure).
- Oracle, Java SE 21 and OpenJFX (JavaFX) documentation.
- PostgreSQL documentation and PostgreSQL JDBC driver documentation.
- Neon documentation (hosted PostgreSQL).
- Apache Maven and jpackage documentation.
- CampusCare project repository: https://github.com/sufiyankhan73397-sudo/CampusCare

1.5 Overview of this Document

Section 2 describes the product, its users and its environment. Section 3 gives the specific requirements: interfaces, functional requirements for each of the seven modules, use cases, non-functional requirements, database requirements and business rules. The appendices contain the traceability matrix, the complaint-status transition table, the list of assumptions that still need confirmation, the seeded master data and the list of suggested diagrams. The table below suggests where each reader should start.

Reader	Recommended sections
Project guide / evaluators	Section 1, Section 3.2 (functional requirements), Section 3.3 (use cases), Appendix A
Developers	Section 2, Section 3.1, Section 3.5, Section 3.6, Appendix B
Testers	Section 3.2, Section 3.4, Section 3.6, Appendix A
Administrators / maintainers	Section 2.4, Section 3.1, Section 3.5, Appendix D

2. Overall Description

2.1 Product Perspective

CampusCare is a new, self-contained product. It is a client-server system with two parts: a JavaFX desktop client that every user runs on a Windows computer, and one shared PostgreSQL database hosted in the cloud on Neon. The client reaches the database over the internet through JDBC. There is no separate application server. The project started with a local SQLite file; it was migrated to the shared PostgreSQL database so that users on different computers see the same data.

[Figure 1: System architecture. Several JavaFX clients (Student, Staff, Admin) connect through JDBC over an encrypted connection to the shared Neon PostgreSQL database; attachments are kept in a local application folder on each client.]

Inside the client the code is organised in layers. A screen may only call the layer below it. Multi-step changes run inside one database transaction: commit on success, rollback on failure.

Layer / package	Responsibility
FXML and JavaFX CSS	Screens, layout and styling
controller	Handles user actions, calls services, updates the screen. Contains no SQL.
service	Business rules, validation, role checks, status transitions and transactions
dao	JDBC data access (all SQL, written as parameterised statements)
model	Entity classes and enumerations (Role, Status, Priority)
util, config, database	Validation, hashing, session, navigation; configuration; connection handling, schema creation and seed data

2.2 Product Functions

The functions are divided into seven modules plus a core/infrastructure part.

Module	Summary	Owner
M1 Authentication and Security	Register (students), login, logout, change password, edit profile, role-based access, session handling, input validation	Sufiyan Khan
M2 Complaint Reporting	Submit a complaint with title, description, category, location, priority and one optional attachment; notify the student and all active admins; auto-assign	Sufiyan Nagori
M3 Complaint Tracking and History	My Complaints list with search and filters, complaint details, attachments list, status history timeline, reopen	Sufiyan Nagori
M4 Staff Resolution	Staff dashboard, assigned complaints, accept, start work, resolve with remarks	Roshan Jat
M5 Notification	In-app notifications for submission, assignment, every staff action, reopen and closure; unread count; mark one or all as read	Roshan Jat
M6 Analytics, Feedback and Reports	Admin analytics, student feedback, feedback list, department rating averages, overall rating, CSV export of complaints	Sumit Yadav
M7 Admin Management	Add users, activate/deactivate users, manage departments, categories and locations, assign complaints to staff, view all complaints	Divyanshu Geel
Core / Infrastructure	Database connection and initializer, configuration, navigation, packaging as EXE	Sufiyan Khan

2.3 User Classes and Characteristics

User class	Description and main privileges	Technical skill
Student	Registers, logs in, submits complaints with an optional attachment, tracks status and history, searches and filters own complaints, receives notifications, reopens a resolved complaint, gives feedback, edits the profile and changes the password.	Basic
Staff	Belongs to one department; sees only complaints assigned to them; accepts, starts work and resolves them; receives notifications.	Basic
Admin	Sees all complaints; assigns or re-assigns complaints; adds users and activates or deactivates them; manages departments, categories and locations; views analytics, feedback and department ratings; exports CSV reports.	Intermediate
Developer / maintainer	Student engineers who build, package and extend the application and maintain the database.	Advanced

A default admin account and one demo staff account per department are created on the first run. Passwords are stored as BCrypt hashes. Only users with ACTIVE status can be assigned complaints.

2.4 Operating Environment

- Client: Windows 10 or Windows 11 computer. The Windows EXE created with jpackage contains the Java runtime, so Java need not be installed separately.
- Network: an internet connection is required, because the database is in the cloud.
- Database: PostgreSQL hosted on Neon, accessed through JDBC.
- Development: Java 21, JavaFX, FXML, JavaFX CSS, Maven, VS Code on Windows 11, Git and GitHub.

2.5 Design and Implementation Constraints

- Language and interface: Java 21 with JavaFX, FXML and JavaFX CSS.
- Layered architecture: controllers contain no SQL; all database access goes through DAO classes.
- Multi-step changes run inside one database transaction (commit on success, rollback on failure).
- The database has exactly the ten tables listed in Section 3.5.
- Passwords are stored only as BCrypt hashes.
- Database credentials are kept in a configuration file that is not committed to GitHub.
- Uploaded attachments are stored under a random file name in a local application folder; only the path is stored in the database.
- Source control: branches main (final stable), develop (integration) and feature/student, feature/staff, feature/admin-dashboard, feature/admin-mgmt. Direct pushes to main are not made.

2.6 User Documentation

- README in the repository: how to build, configure and run the application.
- This SRS, together with the design diagrams and test documents of the project.
- A short user guide for the three roles. Assumption: to be confirmed (A-21).

2.7 Assumptions and Dependencies

The system depends on the following external factors:

- a working internet connection on every client computer;
- availability of the Neon-hosted PostgreSQL database;
- the PostgreSQL JDBC driver and the BCrypt library, obtained through Maven.

Points that the project description does not specify are listed in Appendix C. Each is marked "Assumption: to be confirmed" and is referred to in the text as A-nn.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface

The interface uses JavaFX, FXML and JavaFX CSS with a sidebar and top bar, dashboard cards, tables with search and filters, a status timeline and charts. The screens are:

Screen	Actor	Purpose
Login and Register	All / new student	Sign in; create a student account
Student Dashboard	Student	Counts of pending and completed complaints and recent complaints
Report Issue	Student	Title, description, category, location, priority, one optional attachment; shows the department read-only
My Complaints	Student	Table with text search and status, priority and category filters; buttons Details, Reopen, Give Feedback
Complaint Details and Timeline	Student, Staff, Admin	Complaint data, attachments list and status history timeline
Reopen and Feedback dialogs	Student	Reason for reopening; rating 1 to 5 and comment
Notifications	All	Own notifications, unread count, mark one or all as read
My Profile / Change Password	All	Edit name and phone; change password
Staff Dashboard	Staff	Counts of assigned complaints by status
Assigned Complaints	Staff	List with Accept, Start Work and Resolve actions
Admin Dashboard and Analytics	Admin	Complaints by status, priority, department and category; total users and staff
All Complaints and Assignment	Admin	Search and filter all complaints; assign or re-assign to staff
Manage Users	Admin	Add users; activate or deactivate
Manage Departments, Categories, Locations	Admin	Add, update, activate/deactivate departments; add, update, delete categories and locations
Feedback and Reports	Admin	Feedback list, department and overall ratings, CSV export

3.1.2 Hardware Interfaces

- Client: a standard Windows 10/11 computer with display, keyboard and mouse. Assumption: to be confirmed (A-19), minimum 4 GB RAM and a 1366 x 768 display.
- Server: none to be operated by the project; the database is a managed cloud service.

3.1.3 Software Interfaces

- PostgreSQL on Neon through the PostgreSQL JDBC driver.
- BCrypt library for password hashing.
- Java 21 runtime (bundled), JavaFX libraries, Maven for build, jpackage for the Windows EXE.
- The local file system of the client, for attachments.

3.1.4 Communication Interfaces

- JDBC over TCP/IP to the Neon database through the internet. Assumption: to be confirmed (A-18), the connection is encrypted with TLS.
- No other network protocol is used; the application sends no e-mail or SMS.

3.2 Functional Requirements

Priority: High = essential for the system to be accepted; Medium = important; Low = desirable. Each requirement

is a testable statement. "Active staff member" means a staff user whose status is ACTIVE.

3.2.1 M1 Authentication and Security (owner: Sufiyan Khan)

ID	Description	Input	Processing	Output	Priority
FR-AUTH-01	The system shall allow a new user to register as a student.	Name, email, phone, password	Validate each field (BR-16 to BR-19); check that the email is unique; hash the password with BCrypt; store the user with role STUDENT and status ACTIVE.	Account created and login screen shown, or a field-level error.	High
FR-AUTH-02	The system shall authenticate a user by email and password and open the dashboard of the user's role.	Email, password	Find the user by email; reject a user who is not ACTIVE (A-05); compare with the stored BCrypt hash; create the session.	Student, Staff or Admin dashboard, or a generic "invalid email or password" message.	High
FR-AUTH-03	The system shall allow a logged-in user to log out.	Logout action	Clear the session data.	Login screen.	High
FR-AUTH-04	The system shall allow a logged-in user to change the password.	Current password, new password	Verify the current password (A-09); validate the new password against BR-19; store a new BCrypt hash.	Success message, or the rule that failed.	Medium
FR-AUTH-05	The system shall allow a logged-in user to edit own profile.	Name, phone	Validate (BR-16, BR-18); update the user record (A-16).	Updated profile.	Medium
FR-AUTH-06	The system shall allow each screen and operation only to the roles permitted for it (STUDENT, STAFF, ADMIN).	Logged-in user, requested operation	Check the role in the service layer before the operation runs.	Operation executed, or "access denied" with no data change.	High
FR-AUTH-07	The system shall keep the logged-in user's id, name, role and department in an in-memory session until logout or exit.	Successful login	Store in the session object; clear at logout.	Session data available to all screens.	Medium
FR-AUTH-08	The system shall validate all user input before processing it.	Any form input	Apply required, length and format rules (Section 3.6).	Valid input is processed; otherwise the field is marked with a message and nothing is saved.	High
FR-AUTH-09	The system shall store passwords only as BCrypt hashes.	Password	Hash before saving; never show or log a password.	Hash in users.password.	High

3.2.2 M2 Complaint Reporting (owner: Sufiyan Nagori)

ID	Description	Input	Processing	Output	Priority
FR-REP-01	The system shall allow a student to submit a complaint.	Title, description, category, location, priority, optional attachment	Validate (BR-03); set the owner to the logged-in student; store the complaint with status SUBMITTED, created_at and updated_at.	Confirmation with the complaint number, department, status and assigned staff.	High
FR-REP-02	The system shall offer categories and locations from the database as selection lists.	Form opened	Load the lists; the student cannot type a new value.	Filled lists.	High
FR-REP-03	The system shall take the department of a complaint from its category; the student shall not choose the department.	Selected category	Look up the department of the category and show it read-only on the form.	Department shown on the form and in the details.	High
FR-REP-04	The system shall accept a priority of NORMAL, MEDIUM, HIGH or CRITICAL.	Priority	Reject any other value.	Priority saved.	High

ID	Description	Input	Processing	Output	Priority
FR-REP-05	The system shall accept one optional attachment of type png, jpg, jpeg or pdf, up to 5 MB.	File	Check extension and size; copy the file to a local application folder under a random file name; store file_name and file_path in attachments (A-08).	Attachment listed in the details, or an error message for a rejected file.	Medium
FR-REP-06	The system shall write a history row when a complaint is submitted.	Submitted complaint	Insert a complaint_history row with no old status, new status SUBMITTED and a remark.	Timeline starts with SUBMITTED.	High
FR-REP-07	The system shall notify the student with a confirmation and notify every active admin of the new complaint.	Submitted complaint	Create one notification for the student and one for each active admin.	Notifications stored.	High
FR-REP-08	The system shall automatically assign a submitted complaint to the active staff member of the complaint's department who has the lowest workload.	Submitted complaint	Find active staff of the department; choose the lowest workload (A-01); create an active staff_assignments row; set status ASSIGNED; write a history row; notify the student and the staff member.	Complaint ASSIGNED to the chosen staff member.	High
FR-REP-09	The system shall leave a complaint in status SUBMITTED when its department has no active staff member, so that the admin can assign it manually.	Submitted complaint, no active staff	Skip automatic assignment.	Complaint SUBMITTED and visible to the admin for assignment.	High
FR-REP-10	The system shall save the complaint, history rows, attachment record, assignment and notifications of a submission in one database transaction.	Submit action	Commit all changes together; on any failure roll back everything.	A complete record or no record.	High

3.2.3 M3 Complaint Tracking and History (owner: Sufiyan Nagori)

ID	Description	Input	Processing	Output	Priority
FR-TRK-01	The system shall show a student a My Complaints list that contains only that student's complaints.	Screen opened	Load the complaints of the logged-in student.	Table of complaints.	High
FR-TRK-02	The system shall let the user search the list by text.	Search text	Match the text against the complaint data (A-13).	List reduced to matching complaints.	High
FR-TRK-03	The system shall let the user filter the list by status, priority and category, together with the search text.	Filter values	Apply all selected conditions at the same time.	Filtered list.	High
FR-TRK-04	The system shall show a complaint details screen.	Selected complaint	Load the complaint, its department, assigned staff and names of the people involved.	Details screen.	High
FR-TRK-05	The system shall list the attachments of a complaint and let the user open them.	Selected complaint	Read attachments for the complaint; open the file from the stored path.	Attachment list with links.	Medium
FR-TRK-06	The system shall show the status history of a complaint as a timeline.	Selected complaint	Read the complaint_history rows of the complaint with the name of the user who made each change ("System" if none).	Timeline: old status, new status, remarks, user, date and time.	High

ID	Description	Input	Processing	Output	Priority
FR-TRK-07	The system shall let the student who owns a RESOLVED complaint reopen it with a reason of 5 to 500 characters.	Complaint, reason	Check that the user owns the complaint and the status is RESOLVED; validate the reason; set REOPENED; clear resolved_at; write a history row; notify the student and every active admin.	Status REOPENED.	High
FR-TRK-08	The system shall automatically re-assign a reopened complaint to the staff member who worked on it last, or to the least-loaded active staff member if there is none.	Reopened complaint	Select the staff member (A-02); create the assignment; set ASSIGNED; write a history row; notify. If no staff member is available the status stays REOPENED for the admin.	Complaint ASSIGNED again, or REOPENED waiting for the admin.	High
FR-TRK-09	The system shall restrict visibility: a student sees own complaints, a staff member sees only complaints assigned to them, an admin sees all.	Logged-in user	Filter by role in the service layer.	Only permitted complaints returned.	High
FR-TRK-10	The system shall show on the dashboard the number of pending complaints (SUBMITTED, ASSIGNED, ACCEPTED, REOPENED) and completed complaints (RESOLVED, CLOSED) within the user's visibility.	Dashboard opened	Count by status (A-15).	Count cards.	Medium

3.2.4 M4 Staff Resolution (owner: Roshan Jat)

ID	Description	Input	Processing	Output	Priority
FR-STF-01	The system shall show a staff dashboard with the counts of the complaints assigned to the staff member.	Dashboard opened	Count the staff member's complaints by status.	Count cards.	High
FR-STF-02	The system shall list only the complaints for which the logged-in staff member is the active assignee.	Screen opened	Load complaints through the active staff_assignments row.	Assigned complaints list.	High
FR-STF-03	The system shall allow the active assignee to accept a complaint that has status ASSIGNED.	Complaint, optional remarks	Check assignee and status ASSIGNED; set ACCEPTED; write history; notify the student.	Status ACCEPTED.	High
FR-STF-04	The system shall allow the active assignee to start work on a complaint that has status ASSIGNED or ACCEPTED; accepting first is optional.	Complaint, optional remarks	Check assignee and status; set IN_PROGRESS; write history; notify the student.	Status IN_PROGRESS.	High
FR-STF-05	The system shall allow the active assignee to resolve a complaint that has status IN_PROGRESS, with remarks of 5 to 500 characters.	Complaint, remarks	Check assignee and status; validate remarks; set RESOLVED and resolved_at; write history; notify the student.	Status RESOLVED.	High
FR-STF-06	The system shall refuse any action on a complaint by a user who is not its active assignee.	Staff action	Compare the user with the active assignee.	"Not allowed" message; no change.	High
FR-STF-07	The system shall refuse a status change that is not allowed in Appendix B and shall save the status, history and notification of an allowed change in one transaction.	Staff action	Validate the transition; commit together or roll back.	Updated complaint, or an "invalid status transition" message with no change.	High

3.2.5 M5 Notification (owner: Roshan Jat)

ID	Description	Input	Processing	Output	Priority
FR-NOT-01	The system shall create notifications on submission: a confirmation for the student and a new-complaint notice for every active admin.	Submitted complaint	Insert notification rows.	Notifications stored.	High
FR-NOT-02	The system shall create notifications on every assignment (automatic or by the admin) for the student and for the assigned staff member.	Assignment	Insert notification rows.	Notifications stored.	High
FR-NOT-03	The system shall notify the student after each staff action (accept, start work, resolve).	Staff action	Insert a notification row for the student.	Notification stored.	High
FR-NOT-04	The system shall notify the student and every active admin when a complaint is reopened.	Reopen	Insert notification rows.	Notifications stored.	High
FR-NOT-05	The system shall notify the student and the assigned staff member, including the rating, when a complaint is closed after feedback.	Feedback	Insert notification rows with the rating in the message.	Notifications stored.	High
FR-NOT-06	The system shall show a user the user's own notifications with read or unread state.	Notifications screen	Load notifications of the logged-in user.	Notification list.	High
FR-NOT-07	The system shall show the unread count and let the user mark one or all notifications as read.	Mark action	Update is_read; recount.	Updated list and count.	Medium

3.2.6 M6 Analytics, Feedback and Reports (owner: Sumit Yadav)

ID	Description	Input	Processing	Output	Priority
FR-ANL-01	The system shall show the admin the number of complaints by status and by priority.	Dashboard opened	Count complaints grouped by status and by priority.	Cards and charts.	High
FR-ANL-02	The system shall show the admin the number of complaints by department and by category.	Dashboard opened	Count complaints grouped by category and by the department of the category.	Charts.	High
FR-ANL-03	The system shall show the admin the total number of users and the total number of staff.	Dashboard opened	Count users, and users with role STAFF.	Count cards.	Medium
FR-FBK-01	The system shall allow the student who owns a RESOLVED complaint to give feedback once: a rating from 1 to 5 and an optional comment of up to 500 characters.	Rating, optional comment	Check ownership, status RESOLVED and that no feedback exists; validate; save.	Feedback saved; a second feedback for the complaint is refused.	High
FR-FBK-02	The system shall set the complaint to CLOSED when feedback is saved and shall notify the student and the assigned staff member with the rating. There is no separate close action.	Saved feedback	Set CLOSED; write a history row; create the notifications.	Status CLOSED.	High
FR-FBK-03	The system shall show the admin a list of all feedback.	Screen opened	Load feedback with complaint, rating and comment.	Feedback list.	Medium
FR-FBK-04	The system shall show the admin the average rating of each department and the overall rating.	Screen opened	Compute the averages (A-11).	Department ratings and overall rating.	High
FR-RPT-01	The system shall let the admin export the complaints as a CSV file.	Export action, file location	Read the complaints (A-12); write them to the file.	CSV file saved.	Medium

3.2.7 M7 Admin Management (owner: Divyanshu Geel)

ID	Description	Input	Processing	Output	Priority
FR-ADM-01	The system shall let the admin add users.	Name, email, phone, password, role, department (staff)	Validate (BR-16 to BR-19, BR-20); hash the password; store the user (A-10).	New user in the list.	High
FR-ADM-02	The system shall let the admin activate and deactivate users.	User, action	Change the status of the user.	Updated status.	High
FR-ADM-03	The system shall let the admin add and update departments.	Department data	Validate; save.	Updated department list.	High
FR-ADM-04	The system shall let the admin activate and deactivate departments.	Department, action	Change the status of the department (A-05).	Updated status.	Medium
FR-ADM-05	The system shall let the admin add, update and delete categories; each category belongs to exactly one department.	Category data, department	Validate; save; refuse deletion of a category used by complaints (A-06).	Updated category list.	High
FR-ADM-06	The system shall let the admin add, update and delete locations.	Location data	Validate; save; refuse deletion of a location used by complaints (A-06).	Updated location list.	Medium
FR-ADM-07	The system shall let the admin assign or re-assign a complaint with status SUBMITTED, ASSIGNED, ACCEPTED or REOPENED to an active staff member of the complaint's department.	Complaint, staff member	List the active staff of the department; deactivate earlier assignments; create the new assignment; set ASSIGNED; write history; notify the student and the staff member.	Complaint ASSIGNED to the chosen staff member.	High
FR-ADM-08	The system shall let the admin view all complaints with search, filters, details and history.	Search and filter values	Load all complaints; same list behaviour as FR-TRK-02 and FR-TRK-03.	All-complaints list.	High

3.2.8 Core / Infrastructure (owner: Sufiyan Khan)

ID	Description	Input	Processing	Output	Priority
FR-SYS-01	The system shall, on first run, create missing tables and insert the master data of Appendix D, a default admin account and one demo staff account per department.	Application start	Check the database; insert seed data only when it is missing.	Ready database.	High
FR-SYS-02	The system shall read database credentials from a configuration file that is not committed to the Git repository.	Application start	Load the file; stop with a clear message if it is missing.	Credentials available to the connection code only.	High
FR-SYS-03	The system shall connect to the shared PostgreSQL database through JDBC and shall show a clear message when it cannot connect.	Application start, any database action	Open the connection; handle failure without a crash.	Working connection or a clear error message.	High
FR-SYS-04	The system shall provide screen navigation that depends on the user's role.	Menu selection	Load the matching FXML screen.	Selected screen.	Medium
FR-SYS-05	The system shall be packaged as a Windows EXE with the Java runtime bundled.	Build	Package with Maven and jpackage.	Installable EXE.	Medium

3.3 Use Cases

Actors: Student, Staff, Admin. The System acts automatically in the auto-assignment steps. The Neon database and the local file system are supporting systems and not actors.

[Figure 2: Use case diagram showing the actors Student, Staff, Admin and System, and the use cases UC-01 to UC-20 inside the CampusCare boundary.]

3.3.1 Use Case Summary

ID	Use case	Actor	Related requirements
UC-01	Register	Student (new user)	FR-AUTH-01
UC-02	Login	Student, Staff, Admin	FR-AUTH-02, 06, 07
UC-03	Logout	Student, Staff, Admin	FR-AUTH-03
UC-04	Edit profile / change password	Student, Staff, Admin	FR-AUTH-04, 05
UC-05	Submit complaint (with automatic assignment)	Student, System	FR-REP-01 to 10
UC-06	Search and filter My Complaints	Student	FR-TRK-01 to 03
UC-07	View complaint details, attachments and timeline	Student, Staff, Admin	FR-TRK-04 to 06
UC-08	Reopen a resolved complaint	Student, System	FR-TRK-07, 08
UC-09	Give feedback (closes the complaint)	Student, System	FR-FBK-01, 02
UC-10	View notifications	Student, Staff, Admin	FR-NOT-06, 07
UC-11	View assigned complaints	Staff	FR-STF-01, 02
UC-12	Accept complaint and start work	Staff	FR-STF-03, 04, 06, 07
UC-13	Resolve complaint	Staff	FR-STF-05 to 07
UC-14	Assign or re-assign complaint	Admin	FR-ADM-07
UC-15	Manage users	Admin	FR-ADM-01, 02
UC-16	Manage departments	Admin	FR-ADM-03, 04
UC-17	Manage categories and locations	Admin	FR-ADM-05, 06
UC-18	View analytics	Admin	FR-ANL-01 to 03
UC-19	View feedback and department ratings	Admin	FR-FBK-03, 04
UC-20	Export CSV report	Admin	FR-RPT-01

3.3.2 Detailed Use Cases

UC-01 Register

Actor	Student (new user)
Precondition	The application is running and the database is reachable; the user has no account.
Main flow	1. The user opens Register and enters name, email, phone and password. 2. The system validates every field. 3. The system checks that the email is not already used. 4. The system hashes the password and saves the user with role STUDENT and status ACTIVE. 5. The system shows a success message and the login screen.
Alternate flow	A1. A field is invalid: the system marks the field and does not save. A2. The email already exists: the system shows "email already registered". A3. The database is unreachable: the system shows a connection error.
Postcondition	A new ACTIVE student account exists with a hashed password.

UC-02 Login

Actor	Student, Staff, Admin
Precondition	The user has an ACTIVE account.
Main flow	1. The user enters email and password. 2. The system finds the user and verifies the password hash. 3. The system creates the session with id, name, role and department. 4. The system opens the dashboard of the user's role.
Alternate flow	A1. Wrong email or password: the system shows a generic error and stays on the login screen. A2. The account is not ACTIVE: the system refuses the login (A-05).
Postcondition	A session exists and the role dashboard is shown.

UC-05 Submit Complaint (with Automatic Assignment)

Actor	Student; System (automatic assignment)
Precondition	The student is logged in; categories and locations exist.
Main flow	1. The student opens Report Issue and enters title, description, category, location and priority. 2. The system shows the department of the selected category (read-only). 3. The student optionally chooses one attachment and submits. 4. The system validates the input (title 5 to 100, description 10 to 1000 characters) and the attachment (png, jpg, jpeg, pdf; up to 5 MB). 5. The system saves the complaint with status SUBMITTED, the first history row and the attachment (random file name), and notifies the student and every active admin. 6. The System finds the active staff members of the complaint's department and chooses the one with the lowest workload. 7. The System creates the assignment, sets ASSIGNED, writes a history row and notifies the student and the staff member. 8. All of these changes are committed as one transaction and the system shows the complaint number, department, status and assigned staff.
Alternate flow	A1. Invalid input or attachment: the system marks the problem and saves nothing. A2. The department has no active staff member: steps 6 and 7 are skipped, the complaint stays SUBMITTED and the admin assigns it (UC-14). A3. Any failure while saving: the system rolls back all changes.
Postcondition	A complaint exists with a history entry; its status is ASSIGNED, or SUBMITTED if no staff member was available.

UC-07 View Complaint Details, Attachments and Timeline

Actor	Student, Staff, Admin
Precondition	The user is logged in and may see the complaint (owner, active assignee, or admin).
Main flow	1. The user selects a complaint in a list and chooses Details. 2. The system loads the complaint, its department, assigned staff and attachments. 3. The system loads the history rows of the complaint. 4. The system shows the details, the attachment links and the timeline with old status, new status, remarks, user name and time.
Alternate flow	A1. The user may not see the complaint: the system refuses. A2. An attachment file cannot be opened: the system shows a message (A-08).
Postcondition	No data is changed.

UC-08 Reopen a Resolved Complaint

Actor	Student (owner of the complaint); System (automatic re-assignment)
Precondition	The complaint belongs to the student and has status RESOLVED.
Main flow	1. The student selects the complaint in My Complaints and chooses Reopen. 2. The system asks for a reason of 5 to 500 characters. 3. The student enters the reason and confirms. 4. The system sets REOPENED, clears resolved_at, writes a history row with the reason, and notifies the student and every active admin. 5. The System assigns the complaint to the staff member who worked on it last, or to the least-loaded active staff member if there is none. 6. The System sets ASSIGNED, writes a history row and notifies the student and the staff member.
Alternate flow	A1. The status is not RESOLVED or the user is not the owner: Reopen is refused. A2. The reason is missing or has fewer than 5 characters: the complaint is not reopened. A3. No staff member is available: the complaint stays REOPENED and the admin assigns it (UC-14).
Postcondition	The complaint is ASSIGNED again, or REOPENED waiting for the admin.

UC-09 Give Feedback (Closes the Complaint)

Actor	Student (owner of the complaint); System (closure)
Precondition	The complaint belongs to the student, has status RESOLVED and has no feedback yet.
Main flow	1. The student selects the complaint and chooses Give Feedback. 2. The student selects a rating from 1 to 5 and may type a comment of up to 500 characters. 3. The system validates the input and saves the feedback. 4. The System sets the status to CLOSED and writes a history row. 5. The system notifies the student and the assigned staff member, including the rating.
Alternate flow	A1. Feedback already exists or the status is not RESOLVED: the system refuses. A2. No rating selected: the system asks for one.
Postcondition	One feedback record exists and the complaint is CLOSED. There is no separate close action.

UC-12 Accept Complaint and Start Work

Actor	Staff (active assignee)
Precondition	The complaint is assigned to the logged-in staff member.
Main flow	1. The staff member opens Assigned Complaints and selects a complaint with status ASSIGNED. 2. The staff member may choose Accept and enter optional remarks; the system sets ACCEPTED, writes history and notifies the student. 3. The staff member chooses Start Work (allowed from ASSIGNED or ACCEPTED) and may enter remarks; the system sets IN_PROGRESS, writes history and notifies the student.
Alternate flow	A1. The user is not the active assignee, or the status does not allow the action: the system refuses and changes nothing.
Postcondition	The complaint is IN_PROGRESS with a history entry for each step.

UC-13 Resolve Complaint

Actor	Staff (active assignee)
Precondition	The complaint is assigned to the logged-in staff member and has status IN_PROGRESS.
Main flow	1. The staff member selects the complaint and chooses Resolve. 2. The system asks for remarks of 5 to 500 characters. 3. The staff member enters the remarks and confirms. 4. The system sets RESOLVED and resolved_at, writes the history row and notifies the student.
Alternate flow	A1. Remarks are missing or too short: the system does not resolve. A2. The status is not IN_PROGRESS or the user is not the active assignee: the action is refused.
Postcondition	The complaint is RESOLVED; the student can give feedback or reopen it.

UC-14 Assign or Re-assign Complaint

Actor	Admin
Precondition	The Admin is logged in; the complaint has status SUBMITTED, ASSIGNED, ACCEPTED or REOPENED; the department of the complaint has at least one active staff member.
Main flow	1. The Admin opens All Complaints and selects a complaint. 2. The system lists the active staff members of the complaint's department. 3. The Admin selects a staff member and confirms. 4. In one transaction the system deactivates earlier assignments, creates the new assignment, sets ASSIGNED, writes the history row and notifies the student and the staff member.
Alternate flow	A1. No active staff member in the department: the system shows a message and changes nothing. A2. The status changed in the meantime and no longer allows assignment: the system refuses and reloads the list.
Postcondition	The complaint is ASSIGNED to the chosen staff member and earlier assignments are inactive.

UC-19 View Feedback and Department Ratings

Actor	Admin
Precondition	The Admin is logged in.
Main flow	1. The Admin opens Feedback. 2. The system loads all feedback entries with complaint, rating and comment. 3. The system shows the average rating of each department and the overall rating.
Alternate flow	A1. No feedback exists: the system shows an empty list and no average.
Postcondition	No data is changed.

UC-20 Export CSV Report

Actor	Admin
Precondition	The Admin is logged in.
Main flow	1. The Admin opens Reports and chooses Export CSV. 2. The Admin selects the file location. 3. The system reads the complaints and writes the CSV file. 4. The system shows a confirmation.
Alternate flow	A1. The file cannot be written: the system shows an error and creates no partial file. A2. There are no complaints: the system tells the Admin and writes only the header (A-12).
Postcondition	A CSV file with the complaint data exists at the chosen location.

3.4 Non-Functional Requirements

The figures are measurable targets. Those that the project description does not state are listed in Appendix C (A-17, A-18, A-20) and are to be confirmed.

ID	Category	Requirement	Verification
NFR-01	Performance	The system shall open the dashboard within 3 seconds after a successful login on a normal broadband connection.	Timed test
NFR-02	Performance	The system shall display a list of up to 1,000 complaints within 3 seconds.	Timed test with test data
NFR-03	Performance	The system shall complete a complaint submission, including automatic assignment, within 3 seconds without an attachment and within 10 seconds with a 5 MB attachment.	Timed test
NFR-04	Performance	Database and file operations shall run outside the JavaFX UI thread so that the window never stays unresponsive for more than 1 second.	Manual test
NFR-05	Security	The system shall store 100% of passwords as salted BCrypt hashes; no plain-text password shall appear in the database, logs or screens.	Database and log inspection
NFR-06	Security	The system shall use parameterised statements for 100% of SQL queries.	Code review
NFR-07	Security	Database credentials shall not appear in any file committed to the Git repository.	Repository scan
NFR-08	Security	The connection to the database shall be encrypted (A-18).	Connection settings check
NFR-09	Security	Every service method that changes data shall check the user's role; unauthorised calls shall change no data.	Negative tests per role
NFR-10	Security	Error messages shall not show stack traces, SQL text or passwords.	Manual test of error cases
NFR-11	Reliability	Every operation that changes more than one table shall run in one transaction; after a failure no partial data shall remain.	Fault-injection test
NFR-12	Reliability	When the database cannot be reached, the system shall show an error message within 15 seconds and shall not crash.	Test with network disabled
NFR-13	Reliability	Foreign keys and CHECK constraints shall protect the status, priority, role and rating values and the links between tables.	Schema inspection
NFR-14	Availability	The system shall be available whenever the internet connection and the cloud database are available; it has no offline mode. The target is 99% availability of the application's own components during college hours (A-17).	Operational log
NFR-15	Usability	A first-time student shall be able to submit a complaint within 3 minutes without training (test with at least 5 users).	Usability test
NFR-16	Usability	Every validation error shall be shown next to the field concerned in plain language.	Manual test
NFR-17	Usability	The system shall refuse, with a clear message, any action that the current complaint status does not allow (for example Reopen on a complaint that is not RESOLVED).	Negative tests
NFR-18	Portability	The application shall run on Windows 10 and Windows 11 (64-bit) without a separate Java installation.	Test on both versions
NFR-19	Maintainability	The code shall follow the layered structure; controllers shall contain no SQL.	Code review

ID	Category	Requirement	Verification
NFR-20	Maintainability	Public classes shall carry Javadoc comments (A-17).	Code review
NFR-21	Scalability	The system shall support at least 50 concurrent users and 10,000 complaints without a change of design (A-20).	Load test
NFR-22	Scalability	Each client shall open only a small, limited number of database connections so that many clients can share the cloud database (A-20).	Connection count check
NFR-23	Maintainability	Every status change shall pass through one workflow routine that updates status and updated_at and writes one history row; no status change shall exist without a history row.	Code review and data check

3.5 Database Requirements

The shared PostgreSQL database contains the following ten tables. Names of keys are written as in the project design.

Table	Purpose	Main columns and relationships
departments	Campus departments that resolve complaints	department_id, department_name, description, status
users	All accounts (student, staff, admin)	user_id, name, email, password (hash), phone, role, department_id (staff), status, created_at
categories	Complaint types; each belongs to one department	category_id, category_name, department_id
locations	Places where problems occur	location_id, location_name, building, floor, description
complaints	The complaints	complaint_id, user_id, category_id, location_id, title, description, priority, status, created_at, updated_at, resolved_at
staff_assignments	Assignment of a complaint to a staff member	assignment_id, complaint_id, staff_id, assigned_by, assigned_at, status
complaint_history	Audit trail of status changes	history_id, complaint_id, updated_by, old_status, new_status, remarks, updated_at
attachments	Files attached to complaints (path only)	attachment_id, complaint_id, file_name, file_path, uploaded_by, uploaded_at
notifications	In-app notifications	notification_id, user_id, complaint_id, message, is_read, created_at
feedback	Student feedback after resolution	feedback_id, complaint_id, user_id, rating, comments, created_at

Database requirements. The database shall meet these requirements:

- departments, categories and users are linked by foreign keys: a category belongs to one department, and a staff user belongs to one department;
- complaints refer to the student (users), the category and the location; the department of a complaint is found through its category;
- role is one of STUDENT, STAFF, ADMIN; priority is one of NORMAL, MEDIUM, HIGH, CRITICAL; status is one of the seven statuses; rating is 1 to 5;
- email in users is unique; at most one feedback exists per complaint;
- complaint_history rows are only added, never changed or deleted;
- staff_assignments.status marks the active assignment; assigning a complaint again deactivates the earlier assignments, so a complaint has one active assignee;
- the file itself is not stored in the database, only file_name and file_path.

[Figure 3: Entity-relationship diagram of the ten tables with primary keys, foreign keys and cardinalities.]

3.6 Business Rules

ID	Rule
BR-01	A complaint has exactly one of seven statuses: SUBMITTED, ASSIGNED, ACCEPTED, IN_PROGRESS, RESOLVED, CLOSED, REOPENED. Its priority is NORMAL, MEDIUM, HIGH or CRITICAL.
BR-02	A student submits a complaint; it is stored as SUBMITTED, a history row is written, the student receives a confirmation and every active admin is notified.
BR-03	Title: 5 to 100 characters. Description: 10 to 1000 characters. A category, a location and a priority are required.
BR-04	After submission the system automatically assigns the complaint to the active staff member of the complaint's department with the lowest workload (A-01). If the department has no active staff member, the complaint stays SUBMITTED and the admin assigns it manually.
BR-05	The admin can assign or re-assign a complaint to an active staff member of its department while it is SUBMITTED, ASSIGNED, ACCEPTED or REOPENED. Assignment deactivates earlier assignments, sets the status to ASSIGNED and notifies the student and the staff member.
BR-06	Only the staff member who is the active assignee may act on a complaint.
BR-07	Accept is allowed from ASSIGNED only. Start Work is allowed from ASSIGNED or ACCEPTED (accepting first is optional). Resolve is allowed from IN_PROGRESS only.
BR-08	Resolve requires remarks of 5 to 500 characters. Remarks for Accept and Start Work are optional. Each staff action notifies the student.
BR-09	Feedback (rating 1 to 5, optional comment up to 500 characters) can be given only by the student who owns the complaint, only when the status is RESOLVED, and only once per complaint.
BR-10	Submitting feedback automatically changes the status to CLOSED and notifies the student and the assigned staff member with the rating. There is no separate close action.
BR-11	Only the student who owns a complaint can reopen it, only when it is RESOLVED, and the student must give a reason of 5 to 500 characters. The status becomes REOPENED, resolved_at is cleared, and the student and every active admin are notified.
BR-12	A reopened complaint is automatically re-assigned to the staff member who worked on it last, or to the least-loaded staff member if there is none, which sets the status to ASSIGNED again. If no staff member is available it stays REOPENED for the admin to assign (A-02).
BR-13	Pending complaints are SUBMITTED, ASSIGNED, ACCEPTED and REOPENED (counted on the dashboard). Completed complaints are RESOLVED and CLOSED.
BR-14	Every status change goes through one workflow routine that updates status and updated_at and stores a complaint_history row with old status, new status, remarks, the user who made the change and the time.
BR-15	Each category belongs to exactly one department; the department of a complaint is taken from its category.
BR-16	Name: 3 to 60 characters.
BR-17	Email: valid format and unique (A-04).
BR-18	Phone: valid format (A-03).
BR-19	Password: at least 8 characters with at least one uppercase letter, one lowercase letter and one digit.
BR-20	Roles are STUDENT, STAFF and ADMIN. User and department status is ACTIVE or inactive. A staff member belongs to one department.
BR-21	Only users with status ACTIVE can be assigned complaints.
BR-22	Attachments: only png, jpg, jpeg and pdf; at most 5 MB; stored under a random file name in a local application folder; only the path is stored in the database.
BR-23	Multi-step changes run inside one database transaction: commit on success, rollback on failure.
BR-24	On first run the system creates the master data of Appendix D, a default admin account and one demo staff account per department. Passwords are stored as BCrypt hashes.

4. Appendices

Appendix A: Requirements Traceability Matrix

Module	Requirement IDs	Use cases	Owner
M1 Authentication and Security	FR-AUTH-01 to 09	UC-01 to UC-04	Sufiyan Khan
M2 Complaint Reporting	FR-REP-01 to 10	UC-05	Sufiyan Nagori
M3 Tracking and History	FR-TRK-01 to 10	UC-06, UC-07, UC-08	Sufiyan Nagori
M4 Staff Resolution	FR-STF-01 to 07	UC-11, UC-12, UC-13	Roshan Jat
M5 Notification	FR-NOT-01 to 07	UC-10	Roshan Jat
M6 Analytics, Feedback and Reports	FR-ANL-01 to 03; FR-FBK-01 to 04; FR-RPT-01	UC-09, UC-18, UC-19, UC-20	Sumit Yadav
M7 Admin Management	FR-ADM-01 to 08	UC-14, UC-15, UC-16, UC-17	Divyanshu Geel
Core / Infrastructure	FR-SYS-01 to 05; NFR-01 to NFR-23	all	Sufiyan Khan

Appendix B: Complaint Status Transition Table

From	To	Actor	Condition
(new)	SUBMITTED	Student	complaint submitted
SUBMITTED, ASSIGNED, ACCEPTED, REOPENED	ASSIGNED	Admin	active staff of the complaint's department
SUBMITTED	ASSIGNED	System (auto)	at submit, least-loaded active staff exists
ASSIGNED	ACCEPTED	Staff (assignee)	optional remarks
ASSIGNED, ACCEPTED	IN_PROGRESS	Staff (assignee)	optional remarks
IN_PROGRESS	RESOLVED	Staff (assignee)	remarks 5-500 characters required
RESOLVED	REOPENED	Student (owner)	reason 5-500 characters required
REOPENED	ASSIGNED	System (auto)	last staff, else least-loaded staff
RESOLVED	CLOSED	System (auto)	student submits feedback (rating 1-5)

Any transition that is not in this table is rejected with the message "invalid status transition" and nothing is written. CLOSED has no outgoing transition.

[Figure 4: State diagram of a complaint with the seven statuses, the transitions of the table above, and the actor and condition on each arrow.]

Appendix C: Assumptions and Dependencies

The project description does not specify the following points. Each is an **Assumption: to be confirmed** by the team and the project guide.

ID	Assumption: to be confirmed
A-01	Workload of a staff member is the number of open complaints (not RESOLVED or CLOSED) for which the staff member is the active assignee. How a tie between staff members is resolved is not specified.
A-02	"The staff member who worked on it last" is the most recent assignee of the complaint. If that user is not ACTIVE, the complaint is treated as having none and goes to the least-loaded active staff member.
A-03	A valid phone number has 10 digits.
A-04	Emails are compared without regard to case.

ID	Assumption: to be confirmed
A-05	A user who is not ACTIVE cannot log in. The effect of deactivating a department on its categories and staff is not specified.
A-06	A category or location that is used by complaints cannot be deleted.
A-07	A staff member cannot reject an assignment.
A-08	Attachments are stored in a local folder of the client computer. A user on another computer may therefore be unable to open an attachment. Whether the files will be shared is not decided.
A-09	All three roles may change their password and edit their profile; changing the password requires the current password.
A-10	The admin sets the initial password of a user added by the admin; it must meet the password rule. The admin may add users of any role.
A-11	The average rating of a department is the mean rating of the feedback of all complaints whose category belongs to the department. The overall rating is the mean of all ratings.
A-12	The CSV export contains all complaints with one row per complaint; the exact columns and any filters are not defined.
A-13	Search text is matched against the title and the description of a complaint.
A-14	There is no time limit for reopening, no account lock after failed logins and no session time-out.
A-15	Dashboard counts cover the complaints the user may see: own complaints for a student, assigned complaints for a staff member, all complaints for the admin.
A-16	Profile editing covers name and phone; the email cannot be changed.
A-17	The performance, availability and maintainability figures in Section 3.4 are targets proposed in this document and are to be validated with the guide.
A-18	The connection to the Neon database uses TLS and the standard PostgreSQL port.
A-19	Client minimum: Windows 10/11 64-bit, 4 GB RAM, 1366 x 768 display.
A-20	Target load: 50 concurrent users and 10,000 complaints; each client uses a limited number of database connections.
A-21	A short user guide for the three roles will be written. Default account credentials are given in the README and must be changed before real use.
A-22	The remark text written to the history by automatic and manual assignment is not specified.
A-23	The wording of notification messages is not specified.

Dependencies: internet connection on every client; Neon-hosted PostgreSQL database; PostgreSQL JDBC driver; BCrypt library; Java 21 runtime bundled in the EXE.

Appendix D: Seeded Master Data

Created on the first run (FR-SYS-01).

Item	Values
Departments (7)	IT, Electrical, Maintenance, Library, Hostel, Transport, Security
Categories (14) and their department	Wi-Fi, Computer, Network (IT); Electricity, Lights, Fans (Electrical); Furniture, Maintenance, Classroom Equipment, Other (Maintenance); Library (Library); Hostel (Hostel); Transport (Transport); Security (Security)
Locations (13)	Main Gate, Admin Block, Classroom Block A, Classroom Block B, Computer Lab 1, Library, Boys Hostel, Girls Hostel, Canteen, BUS NO 1, BUS NO 2, BUS NO 3, Playground
Accounts	One default admin and one demo staff account for each of the 7 departments

Appendix E: Suggested Diagrams

The diagrams are to be drawn by the team and inserted in the places marked in this document.

Figure	Diagram	What it should show
Figure 1	System architecture (Section 2.1)	JavaFX clients, JDBC over an encrypted connection, Neon PostgreSQL, local attachment folder
Figure 2	Use case diagram (Section 3.3)	Actors Student, Staff, Admin and System; use cases UC-01 to UC-20; include and extend links
Figure 3	ER diagram (Section 3.5)	Ten tables, primary and foreign keys, cardinalities (for example departments 1-N categories, complaints 1-N complaint_history, complaints 0..1 feedback)
Figure 4	State diagram (Appendix B)	Seven statuses, transitions, actor and condition on each arrow
Figure 5	Class diagram	Layers: controllers, ComplaintService and other services, DAOs, workflow routine, model classes and enumerations, with their relationships
Figure 6	Sequence diagram: submit complaint with automatic assignment	Student, controller, ComplaintService, DAOs, database; validation, transaction, history, notifications, least-loaded staff selection
Figure 7	Sequence diagram: resolve, feedback and reopen	Staff accepts, starts, resolves; student gives feedback (CLOSED) or reopens (REOPENED, then automatic ASSIGNED); history and notifications at each step
Figure 8	Activity diagram: complaint lifecycle	Swim-lanes Student, System, Admin, Staff; decisions for staff availability, feedback and reopen
Figure 9	DFD level 0	CampusCare as one process with external entities Student, Staff, Admin and the data store
Figure 10	DFD level 1	Processes: authenticate, submit and assign, resolve, notify, feedback, analyse, manage master data; data stores for the ten tables